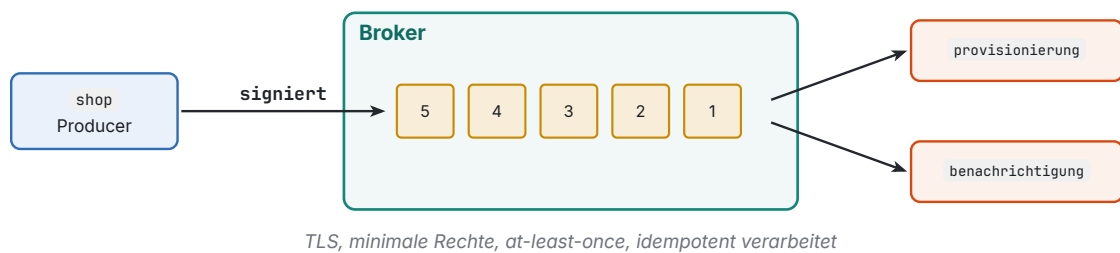


Sichere Messaging-Architekturen

Nachrichtenbasierte Systeme mit Python, NATS JetStream und RabbitMQ: Zustellgarantien, Idempotenz, Rechte und signierte Nachrichten



Inhalt

Bevor es losgeht	3	8.1 Die Bedrohungen	20
Aufbau	3	8.2 TLS und gegenseitige Authentifizierung	20
Konventionen	3	8.3 Identitäten pro Dienst	21
TEIL I · GRUNDLAGEN		9 Minimale Rechte pro Dienst	22
1 Warum Messaging?	4	9.1 Die Rechtematrix entwerfen	22
1.1 Enge und lose Kopplung	4	9.2 NATS: Rechte auf Subjects	22
1.2 Die vier Grundmuster	5	9.3 RabbitMQ: configure, write, read	23
1.3 Befehle und Ereignisse	5	10 Nachrichten signieren und verschlüsseln	24
2 Zustellgarantien	6	10.1 Signieren mit Ed25519	24
2.1 Drei Garantien, eine realistische	6	10.2 Inhalte verschlüsseln	25
2.2 Wo Nachrichten verloren gehen oder sich verdoppeln	6	11 Betrieb und Überwachung	26
2.3 Reihenfolge	6	11.1 Was überwacht werden muss	26
2.4 Giftige Nachrichten und Dead Letters	7	11.2 Nachvollziehbarkeit	26
3 NATS JetStream und RabbitMQ im Überblick	8	11.3 Sicherung und Wiederherstellung	27
3.1 NATS und JetStream	8	11.4 Hochverfügbarkeit in Kürze	27
3.2 RabbitMQ	8	ANHANG	
3.3 Gegenüberstellung	9	A Checkliste	28
3.4 Lokal ausprobieren	9	B Glossar	29
TEIL II · MIT PYTHON		Literaturverzeichnis	30
4 NATS JetStream mit Python	10		
4.1 Stream und Consumer anlegen	10		
4.2 Der Producer	10		
4.3 Der Consumer	11		
5 RabbitMQ mit Python	13		
5.1 Topologie anlegen	13		
5.2 Der Producer	13		
5.3 Der Consumer	14		
6 Nachrichten gestalten	16		
6.1 Ein einheitlicher Umschlag	16		
6.2 Mit Pydantic prüfen, auf beiden Seiten	16		
6.3 Schemata weiterentwickeln	17		
6.4 Was nicht in Nachrichten gehört	17		
7 Zuverlässigkeit in der Anwendung	18		
7.1 Das Problem der zwei Schreibvorgänge	18		
7.2 Transactional Outbox	18		
7.3 Idempotente Consumer	19		
7.4 Wiederholen, aussortieren, wieder einspielen	19		
TEIL III · SICHERHEIT UND BETRIEB			
8 Transport und Authentifizierung	20		

Bevor es losgeht

Sobald mehrere Dienste zusammenarbeiten, stellt sich die Frage, wie sie miteinander sprechen. Direkte HTTP-Aufrufe sind einfach, koppeln die Dienste aber eng: Ist einer langsam oder ausgefallen, bleiben alle stehen, die ihn aufrufen. Nachrichtenbasierte Architekturen entkoppeln die Dienste über einen **Broker**, der Nachrichten zwischenspeichert und verteilt [1], [2]. Das bringt Robustheit, aber auch neue Fragen: Was passiert, wenn eine Nachricht verloren geht oder doppelt ankommt? Wer darf welche Nachrichten senden und lesen? Wie erkennt ein Empfänger, dass eine Nachricht echt ist?

Dieses Handbuch beantwortet diese Fragen für Python mit zwei Brokern, die gleichwertig behandelt werden: **NATS JetStream** und **RabbitMQ**.

Aufbau

- **Teil I, Grundlagen:** wofür Messaging gut ist, welche Muster es gibt, was Zustellgarantien bedeuten, und wie NATS JetStream und RabbitMQ aufgebaut sind.
- **Teil II, mit Python:** Produzenten und Konsumenten mit `nats-py` und `aio-pika`, das Gestalten von Nachrichten und die Muster, die ein System zuverlässig machen (Outbox, idempotente Konsumenten, Dead Letters).
- **Teil III, Sicherheit und Betrieb:** TLS und Authentifizierung, minimale Rechte pro Dienst, signierte und verschlüsselte Nachrichten, Überwachung.
- **Anhang:** Checkliste und Glossar.

Konventionen

Code, Ausgaben und Dateien erscheinen wie in den übrigen Handbüchern. In den Grafiken gilt:

 Producer (sendet)	 Broker, Stream, Queue	 Consumer (empfängt)
 Nachricht	 Schlüssel, Identität	 Angriff / Fehler

Das durchgängige Beispiel ist ein Bestellablauf: Der Dienst `shop` meldet eine eingegangene Bestellung. Der Dienst `provisionierung` richtet daraufhin die bestellten Leistungen ein und meldet den Abschluss, der Dienst `benachrichtigung` informiert den Kunden. Die Ereignisse heißen `bestellung.eingegangen` und `provisionierung.abgeschlossen`. Alle Beispiele verwenden `asyncio`, weil beide Client-Bibliotheken asynchron arbeiten.

MERKE

Das Leitmotiv dieses Buchs: **Jede Nachricht kann verloren gehen, doppelt ankommen, zu spät ankommen oder gefälscht sein**, solange das System nicht ausdrücklich dagegen gebaut ist. Die Kapitel zeigen, wie man jedes dieser vier Probleme löst.

1 Warum Messaging?

1.1 Enge und lose Kopplung



Abb. 1: Links hängt der Erfolg an allen Beteiligten, rechts nur am Broker.

Bei direkten Aufrufen muss jeder Dienst in dem Moment erreichbar sein, in dem ein anderer ihn braucht. Ein Broker entkoppelt in drei Dimensionen: **zeitlich** (der Empfänger kann später verarbeiten), **räumlich** (der Sender muss nicht wissen, wer empfängt und wie viele es sind) und **in der Last** (Lastspitzen werden gepuffert statt weitergereicht) [2], [3].

Der Preis: Das Ergebnis steht nicht sofort fest, Fehler werden später sichtbar, und die Frage „wurde die Nachricht verarbeitet?“ braucht eigene Antworten (Kapitel 2 und 7). Für Abfragen, auf deren Antwort ein Benutzer wartet, bleibt ein direkter API-Aufruf oft die bessere Wahl.

1.2 Die vier Grundmuster

Muster	Verhalten	Beispiel
Queue (Arbeitswarteschlange)	Jede Nachricht wird von genau einem von mehreren gleichartigen Konsumenten verarbeitet (<i>competing consumers</i>). Skalierung durch mehr Konsumenten [4], [5].	Provisionierungsaufträge auf drei Worker verteilen
Publish/Subscribe	Jede Nachricht erreicht alle interessierten Empfängergruppen [6].	Bestellung an Provisionierung und Benachrichtigung
Stream (Log)	Nachrichten werden dauerhaft in Reihenfolge gespeichert. Konsumenten lesen ab einer Position und können zurückspulen [7], [8].	Ereignishistorie, neue Dienste lesen alte Ereignisse nach
Request/Reply	Anfrage über den Broker, Antwort auf einer temporären Rückadresse [9], [10].	Preis abfragen, ohne die Adresse des Preisdienstes zu kennen

In der Praxis werden die Muster kombiniert: Ein Ereignis wird per Pub/Sub an mehrere Gruppen verteilt, innerhalb jeder Gruppe teilen sich mehrere Instanzen die Arbeit wie in einer Queue.

1.3 Befehle und Ereignisse

Nachrichten sind entweder **Befehle** oder **Ereignisse** [11], [12], und der Unterschied prägt das Design:

	Befehl	Ereignis
Bedeutung	„Tu das!“	„Das ist passiert.“
Name	Imperativ: <code>provisionierung.starten</code>	Vergangenheit: <code>bestellung.eingegangen</code>
Empfänger	genau ein zuständiger Dienst	beliebig viele, dem Sender unbekannt
Rechte	Wer darf diesen Auftrag erteilen?	Wer darf behaupten, dass das passiert ist?

Die letzte Zeile ist sicherheitsrelevant: Ein gefälschtes Ereignis `zahlung.eingegangen` kann eine Lieferung auslösen, ohne dass bezahlt wurde. Wer welche Ereignisse veröffentlichen darf, ist deshalb eine der zentralen Fragen in Teil III.

2 Zustellgarantien

Die wichtigste Frage an jedes Messaging-System lautet: Wie oft kommt eine Nachricht an? Die ehrliche Antwort ist bei jedem Broker dieselbe: Ohne Bestätigungen **höchstens einmal**, mit Bestätigungen **mindestens einmal** [3], [13], [14], und genau einmal nie von allein. Wer das verstanden hat, baut Systeme, die mit Verlust und Duplikaten umgehen.

2.1 Drei Garantien, eine realistische

Garantie	Bedeutung	Wann sinnvoll
at-most-once	höchstens einmal: Nachrichten können verloren gehen, kommen aber nie doppelt [13]	Messwerte, bei denen der nächste Wert den verlorenen ersetzt
at-least-once	mindestens einmal: nichts geht verloren, aber Duplikate sind möglich [3], [15]	Standard für Geschäftsvorgänge
exactly-once	genau einmal	über ein Netzwerk nicht allein durch den Broker erreichbar

„Genau einmal“ scheitert an einer einfachen Tatsache: Ein Konsument verarbeitet eine Nachricht und schickt die Bestätigung (*ack*). Geht die Bestätigung verloren oder stürzt der Konsument genau zwischen Verarbeitung und Bestätigung ab, weiß der Broker nicht, ob verarbeitet wurde, und stellt die Nachricht erneut zu [15], [16]. Was sich erreichen lässt, ist **effektiv einmal**: at-least-once-Zustellung plus Empfänger, die Duplikate erkennen und ignorieren (*idempotente Konsumenten*, Kapitel 7) [14], [16].

2.2 Wo Nachrichten verloren gehen oder sich verdoppeln

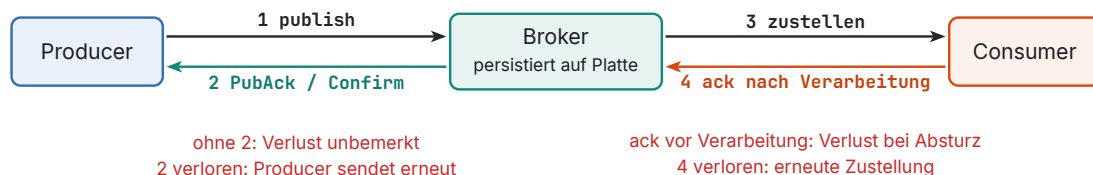


Abb. 2: Zwei Bestätigungen sichern die Strecke ab. Jede Lücke erzeugt Verlust oder Duplikate.

Daraus ergeben sich die Regeln für at-least-once:

1. **Der Broker speichert dauerhaft:** Streams bzw. Queues auf Platte (JetStream `storage: file`, RabbitMQ Quorum Queues und persistente Nachrichten) [14], [17], [18].
2. **Der Producer wartet auf die Bestätigung des Brokers** und sendet bei Ausbleiben erneut [15], [19]. Das kann Duplikate erzeugen, die der Broker über eine Nachrichten-ID erkennt (Kapitel 4 und 5).
3. **Der Consumer bestätigt erst nach erfolgreicher Verarbeitung**, nie beim Empfang. Nicht bestätigte Nachrichten stellt der Broker nach einer Frist erneut zu [15], [20].
4. **Der Consumer ist idempotent**, weil Wiederholungen trotz allem vorkommen [14].

2.3 Reihenfolge

Beide Broker halten die Reihenfolge innerhalb eines Streams bzw. einer Queue ein, **solange genau ein Konsument liest und nichts wiederholt wird**. Mit mehreren parallelen Konsumenten oder nach einer erneuten Zustellung kann eine spätere Nachricht vor einer früheren verarbeitet werden [16], [21]. Systeme sollten deshalb so gebaut sein, dass sie Reihenfolge nicht voraussetzen: Ereignisse tragen einen Zeitstempel oder eine Versionsnummer, und der Empfänger ignoriert veraltete Zustände. Wo Reihenfolge zwingend ist (etwa pro Bestellung), verarbeitet man pro Schlüssel sequenziell.

2.4 Giftige Nachrichten und Dead Letters

Eine Nachricht, deren Verarbeitung immer wieder fehlschlägt (ungültiges Format, Fehler im Code, dauerhaft fehlende Daten), würde ohne Begrenzung endlos erneut zugestellt, Ressourcen verbrauchen und andere Nachrichten blockieren [20], [22]. Dagegen helfen drei Bausteine:

Baustein	Wirkung
Höchstzahl an Zustellungen	JetStream: <code>max_deliver</code> am Consumer [20]. RabbitMQ: <code>x-delivery-limit</code> an Quorum Queues (seit 4.0 standardmäßig 20) [18].
Verzögerte Wiederholung	vorübergehende Fehler (Partner nicht erreichbar) mit wachsendem Abstand erneut versuchen, statt sofort [20]
Dead-Letter-Ziel	Nachrichten, die endgültig scheitern, landen in einer eigenen Queue bzw. einem eigenen Subject, werden überwacht, untersucht und nach der Korrektur gezielt erneut eingespielt [23], [24], [25]

Ein wichtiges Detail: Die Verarbeitung muss zwischen **vorübergehenden** Fehlern (erneut versuchen) und **dauerhaften** Fehlern (sofort als Dead Letter aussortieren) unterscheiden [20]. Eine Nachricht mit ungültiger Signatur wird durch zwanzig Wiederholungen nicht gültig.

3 NATS JetStream und RabbitMQ im Überblick

Beide Broker lösen dieselben Aufgaben, denken aber in unterschiedlichen Begriffen. Wer die beiden Modelle nebeneinander sieht, kann Konzepte übertragen und bewusst wählen.

3.1 NATS und JetStream

NATS ist ein sehr schlanker Nachrichtenserver [26]. In seiner Grundform (*Core NATS*) leitet er Nachrichten nur an gerade verbundene Empfänger weiter, ohne sie zu speichern (*at-most-once*) [13]. **JetStream** ist die eingebaute Persistenzschicht darüber: Ein **Stream** speichert alle Nachrichten zu bestimmten **Subjects**, **Consumer** sind benannte Lesepositionen auf einem Stream [3].

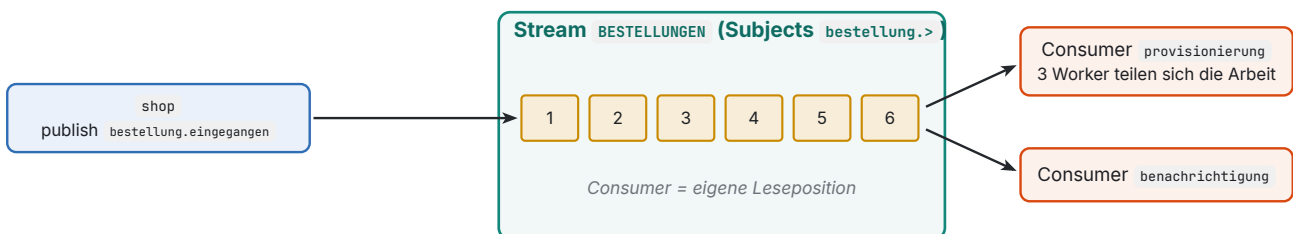


Abb. 3: Ein Stream speichert, jeder Consumer liest unabhängig. Mehrere Instanzen eines Consumers teilen sich die Nachrichten.

- **Subjects** sind hierarchische Namen mit Punkten. `*` steht für genau ein Glied (`bestellung.*`), `>` für ein oder mehrere am Ende (`bestellung.>`) [27].
- Ein **Stream** legt fest, welche Subjects er speichert, wie lange (`max_age`, `max_msgs`, `max_bytes`) und nach welcher Regel (*retention*): `limits` (bis zur Grenze behalten, für Ereignishistorien), `workqueue` (löschen, sobald bestätigt) oder `interest` (löschen, wenn alle Consumer bestätigt haben) [17], [28].
- Ein **Consumer** ist dauerhaft (*durable*) und merkt sich, was bestätigt wurde [7]. *Pull-Consumer* holen Nachrichten aktiv in Stapeln ab, das ist die gängige Form für Worker [29].
- **Deduplizierung** ist eingebaut: Nachrichten mit gleicher `Nats-Msg-Id` innerhalb eines Zeitfensters (`duplicate_window`) speichert der Stream nur einmal [17], [19].

3.2 RabbitMQ

RabbitMQ implementiert das Protokoll AMQP 0-9-1 (und AMQP 1.0) [30]. Producer senden nie direkt an eine Queue, sondern an einen **Exchange**. Der Exchange verteilt anhand von **Bindings** und dem **Routing Key** der Nachricht an Queues, aus denen Consumer lesen [31].

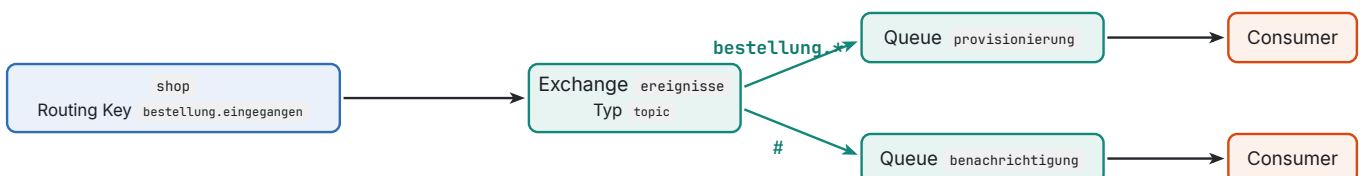


Abb. 4: Der Exchange verteilt nach Bindings an Queues. Jede Queue hat ihre eigenen Consumer.

- **Exchange-Typen:** `direct` (Routing Key muss exakt passen), `topic` (Muster mit `*` für ein Wort und `#` für beliebig viele), `fanout` (an alle gebundenen Queues), `headers` (nach Header-Werten) [31], [32].
- **Queue-Typen:** *Quorum Queues* sind repliziert, dauerhaft und die Standardwahl, wenn eine replizierte, hochverfügbare Queue gebraucht wird [18]. Die alte Spiegelung klassischer Queues wurde in 4.0

entfernt, klassische Queues sind nur noch unrepliziert [33], [34]. *Streams* bieten Log-Semantik wie JetStream [8].

- **Virtual Hosts** (vhosts) trennen Anwendungen oder Umgebungen innerhalb eines Servers logisch voneinander, samt Rechten [35].

3.3 Gegenüberstellung

	NATS JetStream	RabbitMQ
Adressierung	Subject (<code>bestellung.eingegangen</code>) [27]	Exchange + Routing Key -> Queue [31]
Speicher	Stream (Log) mit Consumer-Positionen [3]	Queue (Nachricht weg nach ack), optional Streams [8]
Wiederholtes Lesen	ja, Consumer ab beliebiger Position [7]	nur mit Streams [8]
Deduplizierung beim Senden	eingebaut (<code>Nats-Msg-Id</code>) [19]	nur bei Streams eingebaut, sonst im Consumer lösen [8], [14]
Grenze wiederholter Zustellung	<code>max_deliver</code> am Consumer [20]	<code>x-delivery-limit</code> (Quorum Queue, Standard 20) [18]
Dead Letters	über Advisories oder in der Anwendung [25]	eingebaut: Dead-Letter-Exchange [24]
Standard-Maximalgröße einer Nachricht	1 MB (<code>max_payload</code>) [36]	16 MiB (seit 4.0) [34]
Mandanten-Trennung	Accounts [37]	Virtual Hosts [35]
Rechte	Publish/Subscribe pro Subject [38]	configure/write/read pro Resource, Topic-Rechte [39]
Betrieb	ein kleines Binary, sehr ressourcenschonend [26]	Erlang-basiert [40], umfangreiche Verwaltungsoberfläche [41]
Python-Client	<code>nats-py</code> [42]	<code>aio-pika</code> (asynchron) [43], <code>pika</code> (synchron) [43], [44]

Beide sind für die Beispiele dieses Buchs gleichermaßen geeignet. NATS spielt seine Stärken bei hohem Durchsatz, Streams mit Wiederholung und sehr geringem Betriebsaufwand aus, RabbitMQ bei komplexem Routing, eingebauten Dead Letters und seiner ausgereiften Verwaltungsoberfläche.

3.4 Lokal ausprobieren

`compose.yaml` (nur für die Entwicklung, ohne TLS und Rechte)

```
services:
  nats:
    image: nats:2
    command: ["-js", "-sd", "/data", "-m", "8222"]
    ports: ["127.0.0.1:4222:4222", "127.0.0.1:8222:8222"]
    volumes: [natsdaten:/data]
  rabbitmq:
    image: rabbitmq:4-management
    ports: ["127.0.0.1:5672:5672", "127.0.0.1:15672:15672"]
    volumes: [rabbitdaten:/var/lib/rabbitmq]
volumes:
  natsdaten:
  rabbitdaten:
```

Die Verwaltungsoberfläche von RabbitMQ ist dann unter `http://localhost:15672` erreichbar, für NATS gibt es das Kommandozeilenwerkzeug `nats` (`brew install nats-io/nats-tools/nats`). Wie die Absicherung für den Betrieb aussieht, zeigt Teil III.

4 NATS JetStream mit Python

Der offizielle Python-Client heißt `nats-py` [45] (`uv add nats-py`). Er arbeitet vollständig mit `asyncio` [42]. Dieses Kapitel baut den Producer im `shop` und den Consumer in `provisionierung`, zunächst ohne Sicherheitsfunktionen, die Teil III ergänzt.

4.1 Stream und Consumer anlegen

Streams und Consumer sind Infrastruktur und gehören nicht in den Startcode jedes Dienstes. Legt jeder Dienst sie selbst an, braucht er dafür weitreichende Rechte (Kapitel 9). Besser: Einmalig beim Deployment mit dem `nats`-Werkzeug oder einem eigenen Einrichtungsskript unter einem Administrationskonto:

```
nats stream add BESTELLUNGEN --subjects "bestellung.>" --storage file \
  --retention limits --max-age 30d --dupe-window 2m --replicas 1 --defaults

nats consumer add BESTELLUNGEN provisionierung --pull --filter bestellung.eingegangen \
  --ack explicit --max-deliver 5 --wait 60s --deliver all --defaults
```

Einstellung	Bedeutung
<code>--storage file</code>	auf Platte speichern (sonst Arbeitsspeicher, Verlust bei Neustart) [46]
<code>--retention limits</code> , <code>--max-age 30d</code>	Nachrichten 30 Tage aufheben, auch nach Verarbeitung (Historie, Nachverarbeitung) [28], [47]
<code>--dupe-window 2m</code>	Nachrichten mit gleicher <code>Nats-Msg-Id</code> innerhalb von zwei Minuten nur einmal speichern [48]
<code>--replicas</code>	Anzahl der Kopien im Cluster (1 bei einem Server, 3 im Cluster) [49]
<code>--ack explicit</code>	jede Nachricht einzeln bestätigen [20]
<code>--max-deliver 5</code>	höchstens fünfmal zustellen, dann aufgeben [20]
<code>--wait 60s</code>	<i>ack wait</i> : ohne Bestätigung nach 60 Sekunden erneut zustellen [20]

Die Optionen entsprechen den Flags des `nats`-Werkzeugs [50], [51].

4.2 Der Producer

`shop/nachrichten.py`

```
import json, uuid
import nats
from nats.js.errors import NoStreamResponseError

async def verbinden() -> nats.NATS:
    return await nats.connect(
        servers=["nats://nats.example.com:4222"],
        name="shop", # erscheint in der Überwachung
        max_reconnect_attempts=-1, # bei Netzproblemen unbegrenzt neu verbinden
    )

async def bestellung_melden(nc: nats.NATS, bestellung_id: str, positionen: list[dict]) -> None:
    js = nc.jetstream()
    nachricht_id = str(uuid.uuid4())
    daten = json.dumps({"bestellung_id": bestellung_id, "positionen": positionen}).encode()
    ack = await js.publish(
        "bestellung.eingegangen",
```

```

    daten,
    headers={"Nats-Msg-Id": nachricht_id}, # Deduplizierung im Stream
    timeout=5,
)
if ack.duplicate:
    print(f"bereits gespeichert: {nachricht_id}")

```

`js.publish` wartet auf die Bestätigung (*PubAck*) des Streams [19], [52], erst dann hat der Stream die Nachricht gespeichert. Bleibt sie aus, wirft der Aufruf eine Ausnahme (Zeitüberschreitung oder `NoStreamResponseError`, wenn kein Stream das Subject speichert [19], [52]). Der Aufrufer muss dann erneut senden, **mit derselben** `Nats-Msg-Id`, damit ein doch angekommenes Original nicht doppelt gespeichert wird [19]. Deshalb wird die ID einmal erzeugt und bei Wiederholungen beibehalten, idealerweise aus der Outbox (Kapitel 7). Eine bestätigte Nachricht ist allerdings nicht zwingend schon auf der Platte: Der Server ruft `fsync` standardmäßig nur alle zwei Minuten auf und bestätigt sofort, nach einem Absturz des Betriebssystems können kürzlich bestätigte Nachrichten fehlen [53].

ACHTUNG

Ein einfaches `nc.publish(...)` (ohne `jetstream()`) ist Core NATS: Es gibt keine Bestätigung, und ist gerade kein Empfänger verbunden, ist die Nachricht weg [19], [54]. Für Geschäftsvorgänge immer `js.publish` verwenden.

4.3 Der Consumer

provisionierung/worker.py

```

import asyncio, json, logging
import nats
from nats.errors import TimeoutError as NatsTimeout

log = logging.getLogger("provisionierung")

class VoruebergewendenderFehler(Exception): ... # z.B. Partnerdienst nicht erreichbar
class DauerhafterFehler(Exception): ... # z.B. ungültige Nachricht

async def verarbeiten(daten: dict) -> None: ... # eigentliche Arbeit, idempotent (Kapitel 7)

async def worker(stopp: asyncio.Event) -> None:
    nc = await nats.connect(servers=["nats://nats.example.com:4222"], name="provisionierung")
    js = nc.jetstream()
    psub = await js.pull_subscribe_bind("provisionierung", stream="BESTELLUNGEN")
    while not stopp.is_set():
        try:
            nachrichten = await psub.fetch(batch=10, timeout=5)
        except NatsTimeout:
            continue # nichts da, weiter warten
        for msg in nachrichten:
            versuch = msg.metadata.num_delivered
            try:
                await verarbeiten(json.loads(msg.data))
                await msg.ack()
            except VoruebergewendenderFehler:
                await msg.nak(delay=min(2 ** versuch, 300)) # wachsender Abstand
            except (DauerhafterFehler, json.JSONDecodeError):
                log.error("aussortiert: %s", msg.headers)
                await msg.term() # nie wieder zustellen
    await nc.drain() # offene Arbeit sauber beenden

```

Aufruf	Wirkung
<code>msg.ack()</code>	erfolgreich verarbeitet, der Consumer rückt vor [20]
<code>msg.nak(delay=...)</code>	fehlgeschlagen, nach der Verzögerung erneut zustellen (zählt als Zustellversuch) [20], [52]
<code>msg.in_progress()</code>	noch in Arbeit: setzt die Frist <code>ack_wait</code> zurück, für lange Verarbeitungen [20], [52]
<code>msg.term()</code>	endgültig aufgeben, keine weitere Zustellung [20], [52]
<code>msg.metadata.num_delivered</code>	wie oft diese Nachricht schon zugestellt wurde [52]

`pull_subscribe_bind` bindet sich an den bereits angelegten Consumer, statt ihn selbst zu erzeugen [52]. Ein leerer Abruf ist kein Fehler: `fetch` liefert dann nach Ablauf des Timeouts nichts zurück (die Bibliothek wirft dazu einen `TimeoutError`), und der Worker wartet einfach weiter [29], [52]. Starten mehrere Instanzen des Workers mit demselben Consumer-Namen, verteilt JetStream die Nachrichten automatisch auf sie [55]. Am Ende beendet `drain()` die Verbindung geordnet: Es lässt bereits empfangene Nachrichten fertig verarbeiten und sendet ausstehende Veröffentlichungen ab, bevor die Verbindung schließt. Bestätigen muss der Worker selbst, `drain()` bestätigt keine Nachrichten [56].

Dead Letters in NATS

JetStream hat keinen eingebauten Dead-Letter-Speicher [57]. Zwei übliche Wege: Die Anwendung veröffentlicht eine aussortierte Nachricht vor dem `term()` selbst auf ein eigenes Subject wie `dlq.provisionierung` (das ein eigener Stream speichert), oder ein Überwachungsdienst lauscht auf die Meldungen, die JetStream beim Erreichen von `max_deliver` verschickt (*Advisories* unter `$JS.EVENT.ADVISORY.CONSUMER.MAX_DELIVERIES.>` [58]), und holt die betroffene Nachricht anhand ihrer Sequenznummer aus dem Stream [57]. Der erste Weg ist einfacher und deckt beide Fälle ab, wenn der Worker bei `num_delivered == max_deliver` selbst aussortiert.

5 RabbitMQ mit Python

Für RabbitMQ gibt es zwei verbreitete Python-Clients: `pika` (synchron) [59] und `aio-pika` (asynchron, `uv add aio-pika`) [43]. Dieses Kapitel verwendet `aio-pika`, damit die Beispiele denen für NATS entsprechen. Aufgebaut wird dieselbe Strecke: `shop` sendet, `provisionierung` verarbeitet, gescheiterte Nachrichten landen in einer Dead-Letter-Queue.

5.1 Topologie anlegen

Wie bei NATS gehören Exchanges, Queues und Bindings nicht in den Startcode der Dienste, sondern in die Einrichtung. RabbitMQ kann die gesamte Topologie beim Start aus einer Definitionsdatei laden (`definitions.import_backend` und `definitions.local_path` in `rabbitmq.conf`) [60], alternativ legt ein Einrichtungsskript mit Administratorrechten sie an:

`einrichtung/rabbitmq_topologie.py`

```
import aio_pika

async def einrichten(url: str) -> None:
    async with await aio_pika.connect_robust(url) as verbindung:
        kanal = await verbindung.channel()
        ereignisse = await kanal.declare_exchange("ereignisse", aio_pika.ExchangeType.TOPIC,
        durable=True)
        dlx = await kanal.declare_exchange("dlx", aio_pika.ExchangeType.DIRECT, durable=True)

        queue = await kanal.declare_queue("provisionierung", durable=True, arguments={
            "x-queue-type": "quorum",
            "x-delivery-limit": 5, # nach 5 Zustellungen aussortieren
            "x-dead-letter-exchange": "dlx",
            "x-dead-letter-routing-key": "provisionierung",
        })
        await queue.bind(ereignisse, routing_key="bestellung.eingegangen")

        dlq = await kanal.declare_queue("provisionierung.dlq", durable=True,
        arguments={"x-queue-type": "quorum"})
        await dlq.bind(dlx, routing_key="provisionierung")
```

Die Nachricht durchläuft damit: Exchange `ereignisse` -> Queue `provisionierung` -> (nach fünf gescheiterten Zustellungen oder einem `reject` ohne Requeue) Exchange `dlx` -> Queue `provisionierung.dlq` [18], [24].

5.2 Der Producer

`shop/nachrichten.py`

```
import json, uuid
from datetime import datetime, timezone
import aio_pika

async def verbinden(url: str) -> aio_pika.abc.AbstractRobustConnection:
    return await aio_pika.connect_robust(url, client_properties={"connection_name": "shop"})

async def bestellung_melden(verbindung, bestellung_id: str, positionen: list[dict]) -> None:
    kanal = await verbindung.channel(publisher_confirms=True, on_return_raises=True)
    ereignisse = await kanal.get_exchange("ereignisse", ensure=False) # nicht selbst anlegen
    nachricht = aio_pika.Message(
```

```

body=json.dumps({"bestellung_id": bestellung_id, "positionen": positionen}).encode(),
content_type="application/json",
delivery_mode=aio_pika.DeliveryMode.PERSISTENT,          # auf Platte speichern
message_id=str(uuid.uuid4()),                          # für Deduplizierung beim
Empfänger
timestamp=datetime.now(timezone.utc),
type="bestellung.eingegangen",
)
await ereignisse.publish(nachricht, routing_key="bestellung.eingegangen",
                        mandatory=True, timeout=5)

```

- **Publisher Confirms:** Mit `publisher_confirms=True` (bei `aio-pika` der Standard) wartet `publish` auf die Bestätigung des Brokers, dass die Nachricht angenommen und bei Quorum Queues repliziert gespeichert wurde [15], [61]. Bleibt sie aus, gibt es eine Ausnahme und der Producer sendet erneut.
- `mandatory=True`: Passt keine Queue zum Routing Key, meldet RabbitMQ die Nachricht als unzustellbar zurück, statt sie stillschweigend zu verwerfen [62]. Mit `on_return_raises=True` am Kanal wird daraus eine Ausnahme beim `publish` [61]. Ohne beides würde ein Tippfehler im Routing Key zu unbemerktem Datenverlust führen.
- `message_id`: RabbitMQ dedupliziert nicht selbst. Die ID (eine frei wählbare Nachrichteneigenschaft [62]) ermöglicht dem Consumer, Duplikate zu erkennen (Kapitel 7).

5.3 Der Consumer

provisionierung/worker.py

```

import asyncio, json, logging
import aio_pika

log = logging.getLogger("provisionierung")
# verarbeiten(), VoruebergewendenderFehler, DauerhafterFehler wie in Kapitel 4

async def worker(url: str, stopp: asyncio.Event) -> None:
    verbindung = await aio_pika.connect_robust(url, client_properties={"connection_name":
"provisionierung"})
    async with verbindung:
        kanal = await verbindung.channel()
        await kanal.set_qos(prefetch_count=10)          # höchstens 10 unbestätigte
gleichzeitig
        queue = await kanal.get_queue("provisionierung", ensure=False)
        async with queue.iterator() as nachrichten:
            async for msg in nachrichten:
                try:
                    await verarbeiten(json.loads(msg.body))
                    await msg.ack()
                except VoruebergewendenderFehler:
                    await msg.reject(requeue=True)      # erneut zustellen, zählt zum
Limit
                except (DauerhafterFehler, json.JSONDecodeError):
                    log.error("aussortiert: %s", msg.message_id)
                    await msg.reject(requeue=False)     # direkt in die Dead-Letter-Queue
            if stopp.is_set():
                break

```

Aufruf	Wirkung
<code>msg.ack()</code>	erfolgreich verarbeitet, RabbitMQ löscht die Nachricht aus der Queue [15]
<code>msg.reject(requeue=True)</code>	zurück in die Queue, erneute Zustellung (zählt bei Quorum Queues zur <code>x-delivery-limit</code> , ein <code>nack</code> mit Requeue zählt ab RabbitMQ 4.3 nicht mehr dazu) [18]
<code>msg.reject(requeue=False)</code>	endgültig abgelehnt, wird an den Dead-Letter-Exchange weitergeleitet (falls konfiguriert, sonst verworfen) [15], [24]
<code>kanal.set_qos(prefetch_count=10)</code>	Obergrenze für gleichzeitig zugestellte, unbestätigte Nachrichten pro Consumer [15], [63]

Ohne `prefetch_count` (der Wert 0 heißt unbegrenzt) schiebt RabbitMQ einem Consumer beliebig viele Nachrichten zu (bei Quorum Queues höchstens 2000), die dann im Speicher des Consumers liegen und für andere Instanzen blockiert sind [15]. Für den Durchsatz nennt die RabbitMQ-Dokumentation Werte zwischen 100 und 300 als meist optimal [15]. Bei langer Verarbeitungszeit pro Nachricht ist ein kleinerer Wert wie 10 besser, damit sich die Arbeit auf mehrere Instanzen verteilt.

TIPP

Anders als JetStream kennt RabbitMQ erst ab Version 4.3 eine Verzögerung für zurückgegebene Nachrichten: Quorum Queues halten sie dann auf Wunsch mit linear wachsendem Abstand zurück (*Delayed Retry* über `x-delayed-retry-type`, `x-delayed-retry-min` und `x-delayed-retry-max`) [18]. Für ältere Versionen legt man eine Warteschlange mit Ablaufzeit (`x-message-ttl`) an, deren Dead-Letter-Ziel wieder die Arbeitsqueue ist [24], [64]: Der Consumer lehnt ab, die Nachricht wartet in der Warteschlange und kehrt nach Ablauf zurück. Für einfache Fälle genügt das sofortige Requeue mit der Obergrenze aus `x-delivery-limit`.

ACHTUNG

Seit RabbitMQ 4.0 haben Quorum Queues standardmäßig eine Zustellobergrenze von 20. Ist kein Dead-Letter-Exchange konfiguriert, werden Nachrichten nach 20 gescheiterten Zustellungen **gelöscht** [18]. Jede Quorum Queue braucht deshalb ein Dead-Letter-Ziel, am besten per Policy für alle Queues eines vhosts [18], [24].

6 Nachrichten gestalten

Eine Nachricht ist ein Vertrag zwischen Diensten, die sich nicht kennen und unabhängig voneinander weiterentwickelt werden. Ein durchdachtes Format macht diesen Vertrag prüfbar, versionierbar und sicher.

6.1 Ein einheitlicher Umschlag

Jede Nachricht besteht aus einem **Umschlag** mit Metadaten und den eigentlichen **Nutzdaten**. Das Format orientiert sich an der Spezifikation *CloudEvents* [65]: `id`, `source`, `type`, `time` und `data` stammen von dort (das Pflichtattribut `specversion` ist der Kürze halber weggelassen). `version` und `korrelation_id` sind eigene Ergänzungen. `korrelation_id` wäre als CloudEvents-Erweiterung so nicht zulässig, denn Attributnamen bestehen dort nur aus Kleinbuchstaben und Ziffern [65].

Nachricht `bestellung.eingegangen`

```
{
  "id": "0b8f6a52-3c1e-4d7a-9f10-6e2d8a4c1b77",
  "type": "bestellung.eingegangen",
  "version": 2,
  "source": "shop",
  "time": "2026-09-24T09:15:02Z",
  "korrelation_id": "b3a1f2e4",
  "data": {
    "bestellung_id": "B-2026-00417",
    "kunde_id": "K-8812",
    "positionen": [{"produkt": "learn", "menge": 25}]
  }
}
```

Feld	Zweck
<code>id</code>	eindeutig pro Nachricht, Grundlage für Deduplizierung (<code>Nats-Msg-Id</code> [66] bzw. <code>message_id</code>) und idempotente Verarbeitung
<code>type</code> , <code>version</code>	was die Nachricht bedeutet und in welcher Schema-Version sie vorliegt
<code>source</code>	welcher Dienst sie erzeugt hat, wird bei signierten Nachrichten geprüft (Kapitel 10)
<code>time</code>	Zeitpunkt des Ereignisses, nicht des Versands [65]. Grundlage für Replay-Schutz und Reihenfolge
<code>korrelation_id</code>	verbindet alle Nachrichten und Logs, die zu einem Vorgang gehören
<code>data</code>	die eigentlichen Nutzdaten

6.2 Mit Pydantic prüfen, auf beiden Seiten

`gemeinsam/ereignisse.py`

```
from datetime import datetime
from typing import Literal
from uuid import UUID
from pydantic import BaseModel, ConfigDict, Field

class Streng(BaseModel):
    model_config = ConfigDict(extra="forbid", frozen=True)
```

```

class Position(Streng):
    produkt: Literal["learn", "assess"]
    menge: int = Field(ge=1, le=10_000)

class BestellungEingegangenV2(Streng):
    bestellung_id: str = Field(pattern=r"^\B-\d{4}-\d{5}$")
    kunde_id: str = Field(pattern=r"^\K-\d{1,8}$")
    positionen: List[Position] = Field(min_length=1, max_length=100)

class Umschlag(Streng):
    id: UUID
    type: Literal["bestellung.eingegangen"]
    version: Literal[2]
    source: str
    time: datetime
    korrelation_id: str = Field(max_length=64)
    data: BestellungEingegangenV2

```

Die Modelle weisen unbekannte Felder ab (`extra="forbid"` löst einen `ValidationError` aus) und sind unveränderlich (`frozen=True`) [67]. Wertebereiche, Muster und Längen prüft `Field` [68]. Der Producer erzeugt Nachrichten ausschließlich über diese Modelle, der Consumer prüft jede eingehende Nachricht mit `Umschlag.model_validate_json(msg.data)` [69], bevor er irgendetwas damit tut. Eine Nachricht, die die Prüfung nicht besteht, ist ein dauerhafter Fehler und wird sofort aussortiert (Kapitel 2), nicht wiederholt.

Die Modelle liegen in einem gemeinsamen Paket, das Producer und Consumer als Abhängigkeit einbinden, oder werden als JSON-Schema aus Pydantic erzeugt (`Umschlag.model_json_schema()` [69]) und für Dienste in anderen Sprachen veröffentlicht.

6.3 Schemata weiterentwickeln

Änderung	Vorgehen
neues optionales Feld	kompatibel: alte Consumer ignorieren es nur, wenn sie nicht <code>extra="forbid"</code> für <code>data</code> verwenden [67]. Deshalb: neue Felder mit neuer <code>version</code> einführen.
Feld umbenennen, entfernen, Bedeutung ändern	neue <code>version</code> . Der Producer sendet eine Zeit lang beide Versionen oder die Consumer verstehen beide, dann wird die alte abgeschaltet.
neues Ereignis	neuer <code>type</code> , kein Umbau bestehender Ereignisse

Die Regel lautet: Consumer werden **vor** Producern aktualisiert [70]. Ein Consumer, der Version 2 und 3 versteht, kann laufen, bevor der Producer auf Version 3 umstellt, umgekehrt nicht.

6.4 Was nicht in Nachrichten gehört

Nachrichten werden kopiert, gespeichert (bei JetStream mit `limits`-Retention wochenlang), in Dead-Letter-Queues aufbewahrt, von Überwachungswerkzeugen angezeigt und in Logs geschrieben. Jedes Feld darin vervielfältigt sich. Deshalb:

- **Keine personenbezogenen Daten, wo eine ID genügt** (Datenminimierung, Art. 5 Abs. 1 lit. c DSGVO [71]). `kunde_id` statt Name und E-Mail-Adresse. Wer die Details braucht, fragt sie beim zuständigen Dienst ab und bekommt nur, was er darf. Das vereinfacht auch die Umsetzung von Löschpflichten [72]: Name und E-Mail-Adresse werden an einer Stelle gelöscht, nicht in wochenlang aufbewahrten Streams.
- **Keine Geheimnisse**, keine Zugangsdaten, keine Tokens.
- **Keine großen Nutzdaten**. Dateien und große Dokumente gehören in einen Objektspeicher. Die Nachricht enthält nur einen Verweis (*Claim Check* [73]). Beide Broker haben Obergrenzen (Kapitel 3), und große Nachrichten bremsen alle anderen aus.
- Wo sensible Inhalte unvermeidlich sind, werden sie Ende-zu-Ende verschlüsselt (Kapitel 10).

7 Zuverlässigkeit in der Anwendung

Die Broker liefern at-least-once, wenn Producer und Consumer richtig bestätigen. Zwei Lücken schließt aber nur die Anwendung selbst: Eine Datenbankänderung und die zugehörige Nachricht müssen **gemeinsam** gelingen oder scheitern, und eine doppelt zugestellte Nachricht darf nicht doppelt wirken.

7.1 Das Problem der zwei Schreibvorgänge

Der Shop speichert eine Bestellung in PostgreSQL und veröffentlicht danach `bestellung.eingegangen`. Was, wenn der Dienst genau dazwischen abstürzt? Die Bestellung existiert, die Nachricht nie, die Provisionierung passiert nicht. Andersherum (erst senden, dann speichern) kann eine Nachricht für eine Bestellung existieren, deren Transaktion scheitert. Eine gemeinsame, verteilte Transaktion über Datenbank und Broker ist in der Praxis keine Option [74].

7.2 Transactional Outbox

Die Lösung: Die Nachricht wird **in derselben Datenbanktransaktion** wie die Bestellung in eine Tabelle `outbox` geschrieben. Ein separater Prozess (*Relay*) liest die Tabelle und veröffentlicht [74]. Beide Schreibvorgänge liegen damit in einer Transaktion, und das Veröffentlichen wird so lange wiederholt, bis es gelingt.

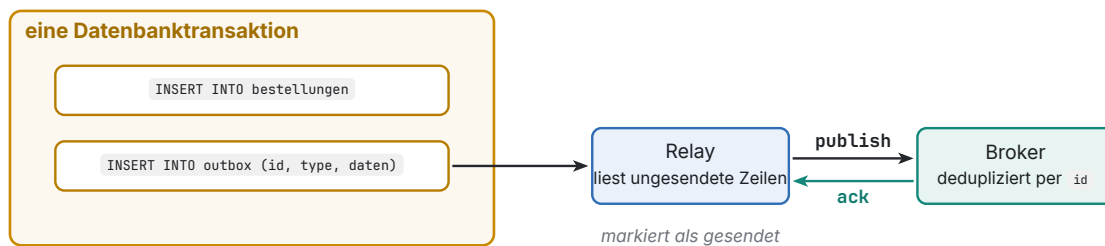


Abb. 5: Bestellung und Nachricht entstehen atomar. Der Relay veröffentlicht mit der Outbox-ID als Nachrichten-ID.

shop/outbox.py

```
async def relay(pool, js, stopp: asyncio.Event) -> None:
    while not stopp.is_set():
        async with pool.connection() as db, db.transaction():
            zeilen = await (await db.execute(
                """SELECT id, type, daten FROM outbox
                WHERE gesendet_am IS NULL ORDER BY erstellt_am
                LIMIT 100 FOR UPDATE SKIP LOCKED""")).fetchall()
            for id_, typ, daten in zeilen:
                await js.publish(typ, daten, headers={"Nats-Msg-Id": str(id_)}, timeout=5)
                await db.execute("UPDATE outbox SET gesendet_am = now() WHERE id = %s", (id_,))
            if not zeilen:
                await asyncio.sleep(1)
```

Stürzt der Relay nach dem Senden, aber vor dem `UPDATE` ab, sendet er die Nachricht beim nächsten Durchlauf erneut. Weil die Outbox-ID als `Nats-Msg-Id` dient, erkennt JetStream das Duplikat innerhalb des Zeitfensters [75]. Dass der Relay eine Nachricht mehrfach veröffentlichen kann, nennt auch die Musterbeschreibung [74]. Bei RabbitMQ wird dieselbe ID als `message_id` gesetzt und erst der Consumer erkennt das Duplikat. `FOR UPDATE SKIP LOCKED` erlaubt mehrere Relay-Instanzen, ohne dass sie sich dieselben Zeilen teilen [76].

7.3 Idempotente Consumer

Das Gegenstück auf der Empfängerseite: Der Consumer merkt sich, welche Nachrichten-IDs er verarbeitet hat, **in derselben Transaktion** wie die eigentliche Wirkung [77]:

provisionierung/verarbeitung.py

```
async def verarbeiten(pool, umschlag: Umschlag) -> None:
    async with pool.connection() as db, db.transaction():
        neu = await (await db.execute(
            """INSERT INTO verarbeitet (nachricht_id, typ) VALUES (%s, %s)
            ON CONFLICT (nachricht_id) DO NOTHING RETURNING nachricht_id""",
            (umschlag.id, umschlag.type))).fetchone()
        if neu is None:
            return # schon verarbeitet: nichts tun, trotzdem
    ack
    await provisionierung_anlegen(db, umschlag.data)
```

Kommt die Nachricht ein zweites Mal, fügt das `INSERT` wegen `ON CONFLICT DO NOTHING` keine Zeile ein und liefert deshalb nichts zurück [78], die Funktion kehrt ohne Wirkung zurück, und der Worker bestätigt die Nachricht. Wird die Transaktion durch einen Fehler abgebrochen, verschwindet auch der Eintrag in `verarbeitet` [79], und die nächste Zustellung versucht es erneut. Die Tabelle `verarbeitet` wird regelmäßig um Einträge bereinigt, die älter sind als die längste mögliche Wiederholungszeit.

Wo keine Datenbank beteiligt ist, lässt sich Idempotenz oft über die Operation selbst erreichen: „Setze Status auf **aktiv**“ ist von Natur aus idempotent, „erhöhe Zähler um 1“ nicht [80].

7.4 Wiederholen, aussortieren, wieder einspielen

Fehlerart	Behandlung
vorübergehend (Netzwerk, Partner überlastet, Sperre in der DB)	erneut zustellen mit wachsendem Abstand (NATS <code>nak(delay)</code> [20], RabbitMQ Warteschlange mit TTL [64], ab 4.3 auch <i>Delayed Retry</i> an Quorum Queues, Kapitel 5)
dauerhaft (ungültiges Schema, ungültige Signatur, unbekannter Typ)	sofort aussortieren [23]: NATS <code>term()</code> [20] plus eigenes Dead-Letter-Subject, RabbitMQ <code>reject(requeue=False)</code> [24]
unklar (Programmfehler)	begrenzte Wiederholungen, danach Dead Letter. Nach dem Bugfix gezielt wieder einspielen.

Dead Letters sind kein Mülleimer, sondern eine Arbeitsliste: Jede Nachricht darin bedeutet einen Vorgang, der nicht abgeschlossen wurde. Die Länge der Dead-Letter-Queues gehört deshalb in die Überwachung (Kapitel 11). Zum Wiedereinspielen liest ein kleines Werkzeug die Dead Letters, prüft sie und veröffentlicht sie mit **derselben** Nachrichten-ID erneut, damit bereits teilweise verarbeitete Vorgänge nicht doppelt wirken.

8 Transport und Authentifizierung

Ein Broker ist ein zentraler Knoten, über den alle Geschäftsvorgänge laufen. Wer ihn unbemerkt mitlesen oder beschreiben kann, kontrolliert das System. Die Grundlage jeder weiteren Maßnahme: verschlüsselte Verbindungen und eine eigene Identität für jeden Dienst.

8.1 Die Bedrohungen

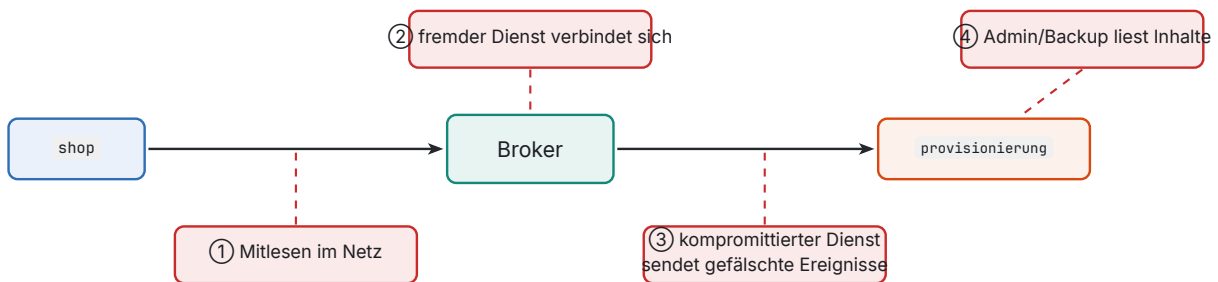


Abb. 6: Vier Angriffswege und ihre Gegenmaßnahmen: TLS (①), Authentifizierung (②), minimale Rechte und Signaturen (③), Verschlüsselung der Inhalte (④).

Dieses Kapitel behandelt ① und ②, Kapitel 9 die Rechte gegen ③, Kapitel 10 Signaturen und Verschlüsselung gegen ③ und ④.

8.2 TLS und gegenseitige Authentifizierung

Beide Broker unterstützen TLS, und beide können zusätzlich ein **Client-Zertifikat** verlangen (mTLS) [81], [82]. Dann weist nicht nur der Server sich gegenüber dem Client aus, sondern auch jeder Dienst gegenüber dem Server. Ein gestohlenen Passwort allein genügt nicht mehr. Die Zertifikate stellt eine eigene, interne Zertifizierungsstelle aus, etwa mit `step-ca` [83] oder für kleine Umgebungen mit `openssl`.

nats-server.conf (Auszug)

```
listen: 0.0.0.0:4222
tls {
  cert_file: "/etc/nats/tls/server.crt"
  key_file:  "/etc/nats/tls/server.key"
  ca_file:   "/etc/nats/tls/ca.crt"
  verify:    true           # Clients müssen ein Zertifikat der internen CA vorlegen
  timeout:   2
}
http: 127.0.0.1:8222        # Überwachung nur lokal (Kapitel 11)
```

rabbitmq.conf (Auszug)

```
listeners.tcp = none                # unverschlüsselten Port 5672 abschalten
listeners.ssl.default = 5671
ssl_options.cacertfile = /etc/rabbitmq/tls/ca.crt
ssl_options.certfile   = /etc/rabbitmq/tls/server.crt
ssl_options.keyfile     = /etc/rabbitmq/tls/server.key
ssl_options.verify      = verify_peer
ssl_options.fail_if_no_peer_cert = true    # mTLS erzwingen
loopback_users.guest   = true            # Standardkonto nur lokal (Standard, trotzdem löschen)
```

In Python wird dasselbe `ssl.SSLContext` für beide Clients verwendet [84]:

gemeinsam/tls.py

```
import ssl

def tls_kontext(dienst: str) -> ssl.SSLContext:
    ctx = ssl.create_default_context(ssl.Purpose.SERVER_AUTH, cafile="/run/secrets/ca.crt")
    ctx.load_cert_chain(f"/run/secrets/{dienst}.crt", f"/run/secrets/{dienst}.key")
    ctx.minimum_version = ssl.TLSVersion.TLSv1_2
    return ctx

# NATS (.creds nur bei JWT-Authentifizierung, sonst user= und password=)
nc = await nats.connect(servers=["tls://nats.example.com:4222"], tls=tls_kontext("shop"),
                        user_credentials="/run/secrets/shop.creds")

# RabbitMQ
verbindung = await aio_pika.connect_robust(
    "amqps://shop@rabbit.example.com:5671/bestellungen", ssl_context=tls_kontext("shop"))
```

8.3 Identitäten pro Dienst

	NATS	RabbitMQ
einfach	Benutzer und Passwort in der Serverkonfiguration (Passwörter als bcrypt-Hash, erzeugt mit <code>nats server passwd</code>) [85]	Benutzer und Passwort in der internen Datenbank (<code>rabbitmqctl add_user</code>) [39]
stärker	NKeys: Schlüsselpaare, der Server kennt nur den öffentlichen Schlüssel [85]	Client-Zertifikat als Identität über das Plugin <code>rabbitmq_auth_mechanism_ssl</code> (Benutzername aus dem Zertifikat) [82]
zentral verwaltet	<i>Decentralized JWT Auth</i> : Operator, Accounts und Benutzer als signierte JWTs, verwaltet mit <code>nsc</code> , Clients erhalten <code>.creds</code> -Dateien [86], [87]	externe Verzeichnisse per LDAP oder OAuth 2.0 (Plugin <code>rabbitmq_auth_backend_oauth2</code>) [88], [89]

Für eine überschaubare Zahl von Diensten genügen Passwörter oder NKeys pro Dienst, kombiniert mit mTLS. Die JWT-basierte Verwaltung von NATS lohnt sich, sobald viele Dienste, Umgebungen oder Mandanten dazukommen, weil sich Benutzer dann ohne Neustart des Servers anlegen und widerrufen lassen [86].

Unabhängig vom Verfahren gilt: **ein Konto pro Dienst**, nie ein gemeinsames Konto für alle, das Standardkonto `guest` von RabbitMQ löschen [90], Zugangsdaten als Docker-Secrets bereitstellen (Docker-Handbuch, Kapitel 12) und die Verwaltungsoberflächen (RabbitMQ Management, NATS-Überwachung) nie öffentlich erreichbar machen [91], [92], sondern nur über VPN oder lokal.

9 Minimale Rechte pro Dienst

Authentifizierung beantwortet, **wer** sich verbindet. Autorisierung legt fest, **was** dieser Dienst darf. Das Ziel ist dasselbe wie bei APIs: Ein kompromittierter Dienst soll nur genau das tun können, was er für seine Aufgabe braucht. Kann der Dienst `benachrichtigung` das Ereignis `zahlung.eingegangen` veröffentlichen, kann ein Angreifer, der ihn übernimmt, Lieferungen ohne Bezahlung auslösen.

9.1 Die Rechtematrix entwerfen

Vor der Konfiguration steht eine Tabelle, die für jeden Dienst festhält, was er senden und empfangen darf:

Dienst	darf senden	darf empfangen
shop	bestellung.eingegangen	(nur Bestätigungen des Brokers)
provisionierung	provisionierung.abgeschlossen, Dead Letters	Consumer <code>provisionierung</code> auf <code>BESTELLUNGEN</code>
benachrichtigung	nichts	Consumer <code>benachrichtigung</code>
Einrichtung (Deployment)	Streams, Consumer, Exchanges, Queues anlegen	nichts

Keiner der Anwendungsdienste darf Streams, Queues oder Exchanges anlegen, ändern oder löschen. Das erledigt ein eigenes Konto beim Deployment (Kapitel 4 und 5).

9.2 NATS: Rechte auf Subjects

In NATS werden Rechte als erlaubte und verbotene Subjects für `publish` und `subscribe` vergeben, mit denselben Platzhaltern `*` und `>` wie bei Subjects [38]. Zwei Besonderheiten von JetStream sind zu beachten: Die Bestätigung eines Streams kommt auf einer `Inbox` zurück (Standard `_INBOX.>`) [38], und ein Pull-Consumer spricht interne API-Subjects an (`$JS.API...`, `$JS.ACK...`) [93], [94]. Die Bestätigungen laufen über Antwort-Subjects mit dem Präfix `$JS.ACK`, die neuere Serverversionen um Domain und Account-Hash erweitern [94]. Deshalb erlaubt die Konfiguration beide Formen. Damit Dienste nicht die Antworten anderer mitlesen können, bekommt jeder ein eigenes Inbox-Präfix [52], [95]:

nats-server.conf (Accounts)

```
accounts {
  APP: {
    jetstream: enabled
    users: [
      { user: shop, password: "$2a$11$...", # bcrypt-Hash
        permissions: {
          publish: { allow: ["bestellung.eingegangen"] }
          subscribe: { allow: ["_INBOX_shop.>"] }
        }
      }
      { user: provisionierung, password: "$2a$11$...",
        permissions: {
          publish: { allow: [
            "$JS.API.CONSUMER.MSG.NEXT.BESTELLUNGEN.provisionierung", # Nachrichten abholen
            "$JS.API.CONSUMER.INFO.BESTELLUNGEN.provisionierung", # an Consumer binden
            "$JS.ACK.BESTELLUNGEN.provisionierung.>", # ack, nak, term
            "$JS.ACK.*.BESTELLUNGEN.provisionierung.>", # dasselbe im neuen
          ] }
          subscribe: { allow: [
            "provisionierung.abgeschlossen",
            "dlq.provisionierung" ] }
        }
      }
    ]
  }
}
```

```

    subscribe: { allow: ["_INBOX_prov.>"] }
  } }
  { user: einrichtung, password: "$2a$11$...",
    permissions: { publish: ["$JS.API.>"], subscribe: ["_INBOX.>"] } }
]
}
SYS: { users: [ { user: admin, password: "$2a$11$..." } ] }
}
system_account: SYS

```

```

nc = await nats.connect(servers=[...], user="shop", password=password,
    inbox_prefix=b"_INBOX_shop", tls=tls_kontext("shop"))

```

Wird ein Recht verweigert, meldet der Server `Permissions Violation` an den Client und schreibt es ins Server-Log [38]. Diese Meldungen gehören in die Überwachung, denn im Betrieb deuten sie entweder auf einen Konfigurationsfehler oder auf einen Angriff hin. **Accounts** trennen zusätzlich ganze Bereiche: Dienste in verschiedenen Accounts sehen die Subjects des anderen überhaupt nicht, außer ein Account exportiert sie ausdrücklich [37]. Getrennte Accounts für Produktion und Test auf demselben Server verhindern, dass ein Testdienst versehentlich echte Bestellungen verarbeitet.

9.3 RabbitMQ: configure, write, read

RabbitMQ vergibt pro Benutzer und vhost drei Rechte, jeweils als regulärer Ausdruck über Namen von Exchanges und Queues [39]:

Recht	erlaubt
configure	Exchanges und Queues anlegen, ändern, löschen
write	in Exchanges veröffentlichen, Queues an Exchanges binden (auf Seiten der Queue)
read	aus Queues lesen, Queues leeren, Bindings (auf Seiten des Exchanges)

```

rabbitmqctl delete_user guest
rabbitmqctl add_vhost bestellungen
rabbitmqctl add_user shop                                # Passwort wird interaktiv abgefragt

#
# vhost      Benutzer      configure  write      read
rabbitmqctl set_permissions -p bestellungen shop      "$*"      "^ereignisse$"  "$*"
rabbitmqctl set_permissions -p bestellungen provisionierung "$*"      "^ereignisse$"
"^provisionierung$"
rabbitmqctl set_permissions -p bestellungen benachrichtigung "$*"      "$*"
"^benachrichtigung$"

# Topic-Rechte: shop darf in "ereignisse" nur mit Routing Keys "bestellung.*" senden
rabbitmqctl set_topic_permissions -p bestellungen shop ereignisse "^bestellung\." "$*"
rabbitmqctl set_topic_permissions -p bestellungen provisionierung ereignisse
"^provisionierung\." "$*"

```

`"^$"` passt auf keinen Namen und entzieht das jeweilige Recht vollständig [39]. Ohne *Topic-Rechte* dürfte `shop` mit Schreibrecht auf `ereignisse` jeden beliebigen Routing Key verwenden, also auch `zahlung.eingegangen`. Erst die Topic-Rechte schränken die Routing Keys ein und schließen damit die Lücke aus der Einleitung [39]. Ein eigener vhost pro Anwendung und Umgebung trennt Bereiche wie die Accounts in NATS [39].

TIPP

Dienste deklarieren ihre Exchanges und Queues in Produktion nicht selbst (Kapitel 5 verwendet `get_exchange(..., ensure=False)` und `get_queue(..., ensure=False)`). Ohne `configure`-Recht würde eine Deklaration ohnehin scheitern [39]. Das hat den Nebeneffekt, dass ein Tippfehler im Queue-Namen nicht stillschweigend eine neue, leere Queue erzeugt.

10 Nachrichten signieren und verschlüsseln

TLS schützt die Strecke, Rechte begrenzen, wer was senden darf. Zwei Lücken bleiben: Der Broker selbst (und jeder mit Zugriff auf ihn, seine Backups oder die Verwaltungsoberfläche) sieht alle Inhalte im Klartext. Und ein Empfänger kann nicht nachprüfen, **welcher Dienst** eine Nachricht erzeugt hat, sondern vertraut darauf, dass der Broker die Rechte korrekt durchsetzt. Signaturen und Verschlüsselung **auf Ebene der Nachricht** schließen diese Lücken Ende-zu-Ende, unabhängig vom Broker.

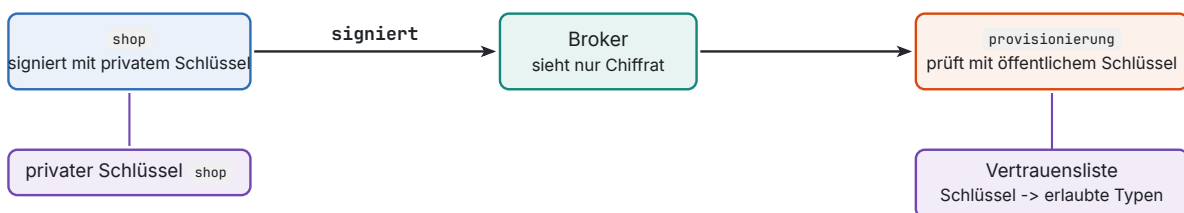


Abb. 7: Die Signatur beweist die Herkunft Ende-zu-Ende. Der Broker muss dafür nicht vertrauenswürdig sein.

10.1 Signieren mit Ed25519

Jeder Producer bekommt ein eigenes Schlüsselpaar. Der private Schlüssel bleibt beim Dienst (als Secret), die öffentlichen Schlüssel aller Producer sind den Consumern bekannt, zusammen mit der Angabe, **welche Ereignistypen** der jeweilige Producer erzeugen darf. Die Bibliothek `cryptography` bringt Ed25519 mit, ein modernes, schnelles Signaturverfahren mit kurzen Schlüsseln [96], [97]:

`gemeinsam/signatur.py`

```

import base64, time
from cryptography.exceptions import InvalidSignature
from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PrivateKey,
Ed25519PublicKey

def _signierbar(typ: str, nachricht_id: str, zeit: str, koerper: bytes) -> bytes:
    # Metadaten mitsignieren, damit sie nicht ausgetauscht werden können
    return b"\n".join([typ.encode(), nachricht_id.encode(), zeit.encode(), koerper])

def signieren(schluesel: Ed25519PrivateKey, schluesel_id: str, typ: str,
nachricht_id: str, koerper: bytes) -> dict[str, str]:
    zeit = str(int(time.time()))
    sig = schluesel.sign(_signierbar(typ, nachricht_id, zeit, koerper))
    return {"X-Schluesel": schluesel_id, "X-Zeit": zeit,
            "X-Signatur": base64.b64encode(sig).decode()}

# Welche Schlüssel-ID darf welche Ereignistypen signieren?
VERTRAUEN: dict[str, tuple[Ed25519PublicKey, set[str]]] = {} # aus Konfiguration geladen

def pruefen(typ: str, nachricht_id: str, koerper: bytes, kopf: dict[str, str],
max_alter: int = 600) -> None:
    eintrag = VERTRAUEN.get(kopf.get("X-Schluesel", ""))
    if eintrag is None or typ not in eintrag[1]:
        raise DauerhafterFehler("unbekannter Schlüssel oder Typ nicht erlaubt")
    if abs(time.time() - int(kopf["X-Zeit"])) > max_alter:
        raise DauerhafterFehler("zu alt: möglicher Replay")
    try:
        eintrag[0].verify(base64.b64decode(kopf["X-Signatur"]),

```

```

        _signierbar(typ, nachricht_id, kopf["X-Zeit"], koerper))
except InvalidSignature:
    raise DauerhafterFehler("Signatur ungültig")

```

Drei Details sind entscheidend:

- **Die Metadaten werden mitsigniert.** Würde nur der Körper signiert, könnte ein Angreifer eine gültig signierte Nachricht mit einem anderen `type` oder einer anderen ID erneut einspielen.
- **Die Vertrauensliste enthält Typen, nicht nur Schlüssel.** Eine gültige Signatur von `benachrichtigung` auf einem Ereignis `zahlung.eingegangen` wird abgelehnt, genau wie die Topic-Rechte aus Kapitel 9, aber unabhängig vom Broker.
- **Replay-Schutz:** Der Zeitstempel begrenzt, wie lange eine abgefangene Nachricht gültig ist. Zusammen mit der Deduplizierung über die Nachrichten-ID im idempotenten Consumer (Kapitel 7) wirkt eine wiederholt eingespielte Nachricht nicht ein zweites Mal. Das Höchstalter muss zu den Wiederholungen passen: Wer Dead Letters nach Tagen wieder einspielt, prüft beim Einspielen oder signiert neu.

Die Prüfung gehört an den Anfang der Verarbeitung, **vor** die Schema-Validierung und vor jeden Datenbankzugriff. Eine ungültige Signatur ist ein dauerhafter Fehler und ein Sicherheitsereignis, das geloggt und gemeldet wird.

10.2 Inhalte verschlüsseln

Enthält eine Nachricht unvermeidlich sensible Inhalte, werden die Nutzdaten zusätzlich verschlüsselt, sodass weder Broker noch Backups noch Verwaltungsoberflächen sie lesen können. AES-GCM verschlüsselt und schützt gleichzeitig gegen Veränderung [98], [99]. Die Nachrichten-ID und der Typ werden als *zusätzliche authentifizierte Daten* (AAD) eingebunden, damit ein Chifftrat nicht in eine andere Nachricht verpflanzt werden kann [100]:

`gemeinsam/verschlueselung.py`

```

import os
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

def verschluesseln(schluesel: bytes, typ: str, nachricht_id: str, klartext: bytes) -> bytes:
    nonce = os.urandom(12)  # pro Nachricht neu, nie
    wiederverwenden
    aad = f"{typ}|{nachricht_id}".encode()
    return nonce + AESGCM(schluesel).encrypt(nonce, klartext, aad)

def entschlueseln(schluesel: bytes, typ: str, nachricht_id: str, daten: bytes) -> bytes:
    nonce, chiffrat = daten[:12], daten[12:]
    return AESGCM(schluesel).decrypt(nonce, chiffrat, f"{typ}|{nachricht_id}".encode())

```

Der Schlüssel (32 zufällige Bytes, `AESGCM.generate_key(bit_length=256)` [100]) wird pro Ereignistyp oder pro Empfängergruppe vergeben und nur den Diensten bereitgestellt, die die Inhalte lesen müssen. Bei zufällig gewählten 96-Bit-Nonces darf ein Schlüssel für höchstens 2^{32} Verschlüsselungen verwendet werden [99]. Für Schlüsselwechsel trägt jede Nachricht eine Schlüssel-ID im Header, und Empfänger halten während einer Übergangszeit den alten und den neuen Schlüssel vor.

ACHTUNG

Kryptographie auf Nachrichtenebene verschiebt das Problem in die Schlüsselverwaltung: Wer die Schlüssel hat, kann alles. Private Schlüssel und AES-Schlüssel sind Secrets wie Datenbankpasswörter (Docker-Secrets, nie im Repository, nie in Logs), brauchen einen Plan für Austausch und Widerruf und sollten pro Umgebung verschieden sein [101], [102]. Für viele Systeme ist **Datensparsamkeit** (Kapitel 6: IDs statt Inhalten) die einfachere und wirksamere Maßnahme als Verschlüsselung.

11 Betrieb und Überwachung

Ein Messaging-System fällt selten laut aus. Typischer ist ein stilles Symptom: Ein Consumer hängt, und Nachrichten stauen sich. Eine Dead-Letter-Queue füllt sich über Tage. Ein falsch konfigurierter Dienst bekommt `Permissions Violation`, ohne dass es jemand bemerkt. Überwachung macht diese Zustände sichtbar, bevor Kunden sie bemerken.

11.1 Was überwacht werden muss

Kennzahl	NATS JetStream	RabbitMQ
Rückstand (unverarbeitete Nachrichten)	<code>num_pending</code> pro Consumer [103]	<code>messages_ready</code> pro Queue [104]
Unbestätigte Nachrichten	<code>num_ack_pending</code> [103]	<code>messages_unacknowledged</code> [104]
Wiederholungen	<code>num_redelivered</code> [103], Advisories zu <code>max_deliver</code>	Redelivery-Rate, <code>x-delivery-count</code> [18]
Dead Letters	Nachrichten im DLQ-Stream	Länge der <code>.dlq</code> -Queues
Consumer aktiv	Anzahl wartender Pull-Anfragen [103]	<code>consumers</code> pro Queue [104]
Rechteverletzungen	Server-Log <code>Permissions Violation</code> [38]	Server-Log <code>access_refused</code> [39]
Ressourcen	Speicherplatz des Stores	Speicher- und Platten-Alarme (<i>alarms</i>) [105]

```
nats consumer info BESTELLUNGEN provisionierung      # Rückstand, ack pending, Redeliveries
nats stream report
rabbitmq-diagnostics check_running
rabbitmqctl list_queues -p bestellungen name messages_ready messages_unacknowledged consumers
```

Für Dashboards und Alarme stellen beide Broker Kennzahlen bereit: NATS über den Überwachungs-Port (`http://127.0.0.1:8222`, z.B. `/jsz` und `/varz`) [36], [91] mit dem offiziellen Prometheus-Exporter [106], RabbitMQ über das mitgelieferte Plugin `rabbitmq_prometheus` (Port 15692) [107]. Beide Endpunkte zeigen Interna und gehören, wie die Verwaltungsoberflächen, nicht ins öffentliche Netz [91].

Sinnvolle Alarme für den Anfang: Rückstand eines Consumers wächst über eine festgelegte Zeit, Dead-Letter-Queue ist nicht leer, ein Consumer hat keine aktive Verbindung, Rechteverletzungen treten auf, Speicherplatz unter 20 Prozent.

11.2 Nachvollziehbarkeit

Die `korrelation_id` aus dem Umschlag (Kapitel 6) verbindet einen Vorgang über alle Dienste: Der Shop setzt sie bei der Bestellung, jeder Dienst übernimmt sie in seine Logs und in alle Folgenachrichten. So lässt sich eine Bestellung vom Eingang bis zur abgeschlossenen Provisionierung verfolgen. Wer verteiltes Tracing mit OpenTelemetry einsetzt, überträgt dessen Kontext (`traceparent`) als Header in der Nachricht [108], [109].

Logs enthalten Nachrichten-ID, Typ, Korrelations-ID und Ergebnis, aber keine Nutzdaten (API-Handbuch, Kapitel 7). Sicherheitsereignisse wie ungültige Signaturen, abgelehnte Nachrichten und Rechteverletzungen werden gesondert protokolliert und gemeldet.

11.3 Sicherung und Wiederherstellung

	NATS JetStream	RabbitMQ
Konfiguration	Stream- und Consumer-Definitionen als Dateien im Repository, angelegt per Skript	Definitionen exportieren: <code>rabbitmqctl export_definitions defs.json</code> , im Repository versionieren (der Export enthält Passwort-Hashes der Benutzer) [60]
Nachrichten	<code>nats backup stream</code> / <code>nats backup restore stream</code> . Drei Replikas schützen vor Knotenausfall, ersetzen aber keine Sicherung [110], [111]	in der Regel nicht gesichert: Queues sind Durchgangsstationen, die Quelle der Wahrheit liegt in den Datenbanken der Dienste. Nachrichten ließen sich nur durch Kopieren des Datenverzeichnisses sichern, nicht aus einem laufenden Knoten [112]

Die Outbox (Kapitel 7) hat hier einen weiteren Vorteil: Weil jede Nachricht zuerst in der Datenbank des Senders steht, lassen sich Nachrichten eines verlorenen Brokers aus den Outbox-Tabellen erneut veröffentlichen.

11.4 Hochverfügbarkeit in Kürze

Ein einzelner Broker ist ein zentraler Ausfallpunkt. Beide Systeme lassen sich als Cluster aus drei Knoten betreiben: NATS mit `--replicas 3` für Streams [113], RabbitMQ mit Quorum Queues, die automatisch auf drei Knoten repliziert werden [18]. Für kleinere Systeme ist ein einzelner, gut überwachter Broker mit schnellem Wiederaufbau (Konfiguration als Code, Outbox als Puffer bei den Producern) oft der pragmatischere Weg.

A Checkliste

ZUVERLÄSSIGKEIT

- ☐ Streams (`storage: file`) bzw. Quorum Queues, persistente Nachrichten
- ☐ Producer wartet auf PubAck bzw. Publisher Confirm
- ☐ Nachrichten-ID pro Nachricht, bei Wiederholung beibehalten
- ☐ RabbitMQ: `mandatory` + `on_return_raises`
- ☐ Ack erst nach erfolgreicher Verarbeitung
- ☐ Höchstzahl an Zustellungen, verzögerte Wiederholung
- ☐ Dead-Letter-Ziel für jede Queue bzw. jeden Consumer
- ☐ Transactional Outbox beim Producer
- ☐ Idempotenter Consumer (Tabelle `verarbeitet`)
- ☐ Keine Annahme über Reihenfolge ohne Absicherung

NACHRICHTEN

- ☐ Einheitlicher Umschlag mit `id`, `type`, `version`, `source`, `time`
- ☐ Pydantic-Validierung auf beiden Seiten, `extra="forbid"`
- ☐ Versionierung, Consumer vor Producern aktualisieren
- ☐ IDs statt personenbezogener Daten, keine Geheimnisse
- ☐ Große Inhalte als Verweis (Claim Check)

SICHERHEIT

- ☐ TLS für alle Verbindungen, unverschlüsselte Ports aus
- ☐ mTLS mit interner CA
- ☐ Eigenes Konto pro Dienst, `guest` gelöscht
- ☐ Rechematrix: senden und empfangen pro Dienst
- ☐ NATS: Subject-Rechte, eigenes Inbox-Präfix, Accounts
- ☐ RabbitMQ: configure `^$` für Dienste, Topic-Rechte, vhosts
- ☐ Topologie nur durch Einrichtungskonto
- ☐ Signaturen mit Typ-Freigabe pro Schlüssel
- ☐ Replay-Schutz: Zeitstempel + Deduplizierung
- ☐ Verschlüsselung für unvermeidbar sensible Inhalte
- ☐ Schlüssel als Secrets, pro Umgebung, mit Wechselplan
- ☐ Verwaltungsoberflächen nur über VPN/lokal

BETRIEB

- ☐ Alarme: Rückstand, Dead Letters, fehlende Consumer
- ☐ Rechtsverletzungen und ungültige Signaturen gemeldet
- ☐ Korrelations-ID in allen Logs und Folgenachrichten
- ☐ Topologie als Code im Repository

B Glossar

Quellen und weiterführende Literatur zu den Begriffen, Standards und Werkzeugen dieses Handbuchs stehen im anschließenden Literaturverzeichnis. Begleitbände: **Secure API Design** (Autorisierung, Logging) und **Docker von Grund auf** (Secrets, Betrieb).

Ack / Nack

Bestätigung bzw. Ablehnung einer empfangenen Nachricht durch den Consumer.

Account (NATS)

Isolierter Bereich mit eigenen Subjects, Streams und Benutzern.

at-least-once

Zustellgarantie: nichts geht verloren, Duplikate sind möglich.

Binding

Regel in RabbitMQ, die eine Queue mit einem Exchange und einem Muster verbindet.

Claim Check

Muster, bei dem große Inhalte extern gespeichert und nur per Verweis verschickt werden.

Consumer

Empfänger von Nachrichten, bei JetStream zugleich eine benannte Leseposition.

Dead Letter

Nachricht, die endgültig nicht verarbeitet werden konnte und aussortiert wurde.

Exchange

Verteiler in RabbitMQ, an den Producer senden.

Idempotenz

Mehrfache Verarbeitung hat dieselbe Wirkung wie einmalige.

Inbox

Temporäres Antwort-Subject in NATS für Bestätigungen und Request/Reply.

JetStream

Persistenzschicht von NATS mit Streams und Consumern.

mTLS

TLS, bei dem sich auch der Client mit einem Zertifikat ausweist.

Outbox

Tabelle, in die Nachrichten in derselben Transaktion wie die Geschäftsdaten geschrieben werden.

Prefetch

Obergrenze unbestätigter Nachrichten pro Consumer in RabbitMQ.

Publisher Confirm

Bestätigung von RabbitMQ an den Producer, dass eine Nachricht angenommen wurde.

PubAck

Bestätigung eines JetStream-Streams, dass eine Nachricht gespeichert wurde.

Quorum Queue

Replizierte, dauerhafte Queue in RabbitMQ auf Basis von Raft.

Replay

Erneutes Einspielen einer (abgefangenen) Nachricht.

Routing Key

Adressangabe einer Nachricht in RabbitMQ, ausgewertet vom Exchange.

Stream

Dauerhaft gespeicherte, geordnete Folge von Nachrichten.

Subject

Hierarchischer Nachrichtenname in NATS, z.B.

`bestellung.eingegangen`.

vhost

Virtueller Host in RabbitMQ: logisch getrennter Bereich mit eigenen Rechten.

Literaturverzeichnis

Die Nummern verweisen auf die Quellenangaben im Text. Online-Quellen wurden am 1. Oktober 2026 abgerufen. Das Verzeichnis enthält nur Quellen, die im Text zitiert werden.

- [1] G. Hohpe und B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.
- [2] B. Woolf, „Messaging“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/Messaging.html>
- [3] NATS Project, „JetStream“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/concepts/jetstream>
- [4] B. Woolf, „Point-to-Point Channel“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/PointToPointChannel.html>
- [5] B. Woolf, „Competing Consumers“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/CompetingConsumers.html>
- [6] B. Woolf, „Publish-Subscribe Channel“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/PublishSubscribeChannel.html>
- [7] NATS Project, „Reading back the stream“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/jetstream/reading-back>
- [8] RabbitMQ Project, „Streams and Superstreams (Partitioned Streams)“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/streams>
- [9] B. Woolf, „Request-Reply“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/RequestReply.html>
- [10] NATS Project, „Request-reply“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/core-nats/request-reply>
- [11] B. Woolf, „Command Message“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/CommandMessage.html>
- [12] B. Woolf, „Event Message“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/EventMessage.html>
- [13] NATS Project, „Core NATS Deep Dive“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/core-nats>
- [14] RabbitMQ Project, „Reliability Guide“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/reliability>
- [15] RabbitMQ Project, „Consumer Acknowledgements and Publisher Confirms“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/confirms>
- [16] NATS Project, „Delivery and acknowledgment“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/jetstream/delivery-and-acknowledgment>
- [17] NATS Project, „Your first stream“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/jetstream/your-first-stream>
- [18] RabbitMQ Project, „Quorum Queues“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/quorum-queues>
- [19] NATS Project, „Publishing“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/jetstream/publishing>
- [20] NATS Project, „Ack responses and redelivery“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/jetstream/acknowledgment>
- [21] RabbitMQ Project, „Queues“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/queues>
- [22] M. Kuratczyk, „RabbitMQ 4.0: New Quorum Queue Features“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/blog/2024/08/28/quorum-queues-in-4.0>
- [23] B. Woolf, „Dead Letter Channel“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/DeadLetterChannel.html>
- [24] RabbitMQ Project, „Dead Letter Exchanges“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/dlx>
- [25] NATS Project, „Advisories & events“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/monitoring/advisories-and-events>

- [26] NATS Project, „nats-io/nats-server“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/nats-io/nats-server>
- [27] NATS Project, „Subjects“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/concepts/subjects>
- [28] NATS Project, „Retention policies“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/jetstream/retention-policies>
- [29] NATS Project, „Pull consumers in depth“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/jetstream/pull-consumers>
- [30] RabbitMQ Project, „AMQP 1.0“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/amqp>
- [31] RabbitMQ Project, „AMQP 0-9-1 Model Explained“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/tutorials/amqp-concepts>
- [32] RabbitMQ Project, „RabbitMQ tutorial – Topics“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/tutorials/tutorial-five-python>
- [33] RabbitMQ Project, „Classic Queue Mirroring (Deprecated)“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/3.13/ha>
- [34] RabbitMQ Project, „RabbitMQ 4.0.1 (Release Notes)“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/rabbitmq/rabbitmq-server/blob/v4.0.x/release-notes/4.0.1.md>
- [35] RabbitMQ Project, „Virtual Hosts“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/vhosts>
- [36] NATS Project, „Configuration“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/reference/config>
- [37] NATS Project, „Accounts and multitenancy“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/security/accounts-and-multitenancy>
- [38] NATS Project, „Authorization“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/security/authorization>
- [39] RabbitMQ Project, „Authentication, Authorisation, Access Control“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/access-control>
- [40] RabbitMQ Project, „Erlang Version Requirements“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/which-erlang>
- [41] RabbitMQ Project, „Management Plugin“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/management>
- [42] NATS Project, „nats-io/nats.py“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/nats-io/nats.py>
- [43] aio-pika contributors, „aio-pika“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.aio-pika.com/>
- [44] RabbitMQ Project, „Clients Libraries and Developer Tools“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/client-libraries/devtools>
- [45] NATS Project, „Download“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://nats.io/download/>
- [46] NATS Project, „Stream and consumer policies“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/jetstream/policies>
- [47] NATS Project, „Shaping the stream“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/jetstream/shaping-the-stream>
- [48] NATS Project, „Streams“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/nats-concepts/jetstream/streams>
- [49] NATS Project, „Surviving node loss“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/jetstream/surviving-node-loss>
- [50] NATS Project, „natscli: cli/stream_command.go“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://github.com/nats-io/natscli/blob/main/cli/stream_command.go
- [51] NATS Project, „natscli: cli/consumer_command.go“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://github.com/nats-io/natscli/blob/main/cli/consumer_command.go
- [52] NATS Project, „nats.py documentation: Modules“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://nats-io.github.io/nats.py/modules.html>
- [53] K. Kingsbury, „Jepsen: NATS 2.12.1“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://jepsen.io/analyses/nats-2.12.1>
- [54] NATS Project, „JetStream“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/nats-concepts/jetstream>
- [55] NATS Project, „Scaling a consumer“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/jetstream/worker-pool>
- [56] NATS Project, „Drain & Shutdown“. Zugriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/resilient-clients/drain-and-shutdown>

- [57] A. Connolly, „Reliable Message Delivery in NATS JetStream“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.synadia.com/blog/jetstream-reliable-delivery-dlq-replay>
- [58] NATS Project, „max deliver“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/reference/jetstream/advisory/max-deliver>
- [59] Pika contributors, „Introduction to Pika“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://pika.readthedocs.io/en/stable/>
- [60] RabbitMQ Project, „Definition Export and Import“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/definitions>
- [61] aio-pika contributors, „API Reference: aio-pika“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.aio-pika.com/apidoc.html>
- [62] RabbitMQ Project, „Publishers“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/publishers>
- [63] RabbitMQ Project, „Consumer Prefetch“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/consumer-prefetch>
- [64] RabbitMQ Project, „Time-To-Live and Expiration“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/ttl>
- [65] CloudEvents Project (CNCF), „CloudEvents - Version 1.0.3-wip“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/cloudevents/spec/blob/main/cloudevents/spec.md>
- [66] NATS Project, „JetStream API headers (Nats-Msg-Id)“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/reference/jetstream/api/headers>
- [67] Pydantic Services Inc., „Configuration (ConfigDict)“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://pydantic.dev/docs/validation/latest/api/pydantic/config/>
- [68] Pydantic Services Inc., „Fields (Field)“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://pydantic.dev/docs/validation/latest/api/pydantic/fields/>
- [69] Pydantic Services Inc., „BaseModel“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://pydantic.dev/docs/validation/latest/api/pydantic/base_model/
- [70] Confluent, Inc., „Schema Evolution and Compatibility for Schema Registry on Confluent Platform“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.confluent.io/platform/current/schema-registry/fundamentals/schema-evolution.html>
- [71] Intersoft Consulting, „Art. 5 GDPR - Principles relating to processing of personal data“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://gdpr-info.eu/art-5-gdpr/>
- [72] Intersoft Consulting, „Art. 17 GDPR - Right to erasure ('right to be forgotten')“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://gdpr-info.eu/art-17-gdpr/>
- [73] G. Hohpe und B. Woolf, „Claim Check“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/StoreInLibrary.html>
- [74] C. Richardson, „Pattern: Transactional outbox“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://microservices.io/patterns/data/transactional-outbox.html>
- [75] NATS Project, „JetStream: Streams“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/jetstream/concepts/streams>
- [76] PostgreSQL Global Development Group, „PostgreSQL Documentation: SELECT (The Locking Clause)“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.postgresql.org/docs/current/sql-select.html>
- [77] C. Richardson, „Pattern: Idempotent Consumer“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://microservices.io/patterns/communication-style/idempotent-consumer.html>
- [78] PostgreSQL Global Development Group, „PostgreSQL Documentation: INSERT“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.postgresql.org/docs/current/sql-insert.html>
- [79] PostgreSQL Global Development Group, „PostgreSQL Documentation: Transactions“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.postgresql.org/docs/current/tutorial-transactions.html>
- [80] G. Hohpe und B. Woolf, „Idempotent Receiver“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/IdempotentReceiver.html>
- [81] NATS Project, „Encryption & TLS“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/security/encryption>
- [82] RabbitMQ Project, „TLS Support“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/ssl>
- [83] Smallstep, „step-ca Certificate Authority Overview“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://smallstep.com/docs/step-ca/>
- [84] Python Software Foundation, „ssl — TLS/SSL wrapper for socket objects“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.python.org/3.13/library/ssl.html>
- [85] NATS Project, „Authentication basics“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/security/authentication-basics>
- [86] NATS Project, „Decentralized authentication“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/security/decentralized-auth>

- [87] NATS Project, „The NATS Ecosystem“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/concepts/ecosystem>
- [88] RabbitMQ Project, „LDAP Support“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/ldap>
- [89] RabbitMQ Project, „OAuth 2.0 Authentication Backend“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/oauth2>
- [90] RabbitMQ Project, „Production Deployment Guidelines“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/production-checklist>
- [91] NATS Project, „Monitoring endpoints“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/monitoring/monitoring-endpoints>
- [92] RabbitMQ Project, „Networking and RabbitMQ“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/networking>
- [93] NATS Project, „Consumer“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/reference/jetstream/api/consumer/>
- [94] NATS Project, „ADR-15: JetStream Subscribe Workflow“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/nats-io/nats-architecture-and-design/blob/main/adr/ADR-15.md>
- [95] NATS Project, „NATS by Example - Private Inbox (CLI)“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://examples.nats.io/examples/auth/private-inbox/cli>
- [96] S. Josefsson und I. Liusvaara, „Edwards-Curve Digital Signature Algorithm (EdDSA)“, Internet Engineering Task Force (IETF), RFC8032, Jan. 2017. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rfc-editor.org/rfc/rfc8032>
- [97] The Cryptography Developers, „Ed25519 signing“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://cryptography.io/en/latest/hazmat/primitives/asymmetric/ed25519/>
- [98] D. McGrew, „An Interface and Algorithms for Authenticated Encryption“, Internet Engineering Task Force (IETF), RFC5116, Jan. 2008. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rfc-editor.org/rfc/rfc5116>
- [99] M. Dworkin, „Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC“, National Institute of Standards and Technology, NIST Special Publication 800-38D, Nov. 2007. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf>
- [100] The Cryptography Developers, „Authenticated encryption“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://cryptography.io/en/latest/hazmat/primitives/aeauth/>
- [101] OWASP Foundation, „Secrets Management - OWASP Cheat Sheet Series“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
- [102] OWASP Foundation, „Key Management - OWASP Cheat Sheet Series“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: https://cheatsheetseries.owasp.org/cheatsheets/Key_Management_Cheat_Sheet.html
- [103] NATS Project, „Consumer Info“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/reference/jetstream/api/consumer/info>
- [104] RabbitMQ Project, „Monitoring“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/monitoring>
- [105] RabbitMQ Project, „Memory and Disk Alarms“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/alarms>
- [106] NATS Project, „prometheus-nats-exporter: A Prometheus exporter for NATS metrics“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://github.com/nats-io/prometheus-nats-exporter>
- [107] RabbitMQ Project, „Monitoring with Prometheus and Grafana“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/prometheus>
- [108] S. Kanzhelev, M. McLean, A. Reitbauer, B. Drutu, N. Molnar, und Y. Shkuro, „Trace Context“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.w3.org/TR/trace-context/>
- [109] OpenTelemetry Authors, „Semantic conventions for messaging spans“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://opentelemetry.io/docs/specs/semconv/messaging/messaging-spans/>
- [110] NATS Project, „Stream backup and restore“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/backup-recovery/stream-backup-restore>
- [111] NATS Project, „Disaster recovery“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/backup-recovery/disaster-recovery>
- [112] RabbitMQ Project, „Backup and Restore“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://www.rabbitmq.com/docs/backup>
- [113] NATS Project, „JetStream in a cluster“. Zugegriffen: 1. Oktober 2026. [Online]. Verfügbar unter: <https://docs.nats.io/learn/topologies/jetstream-in-a-cluster>